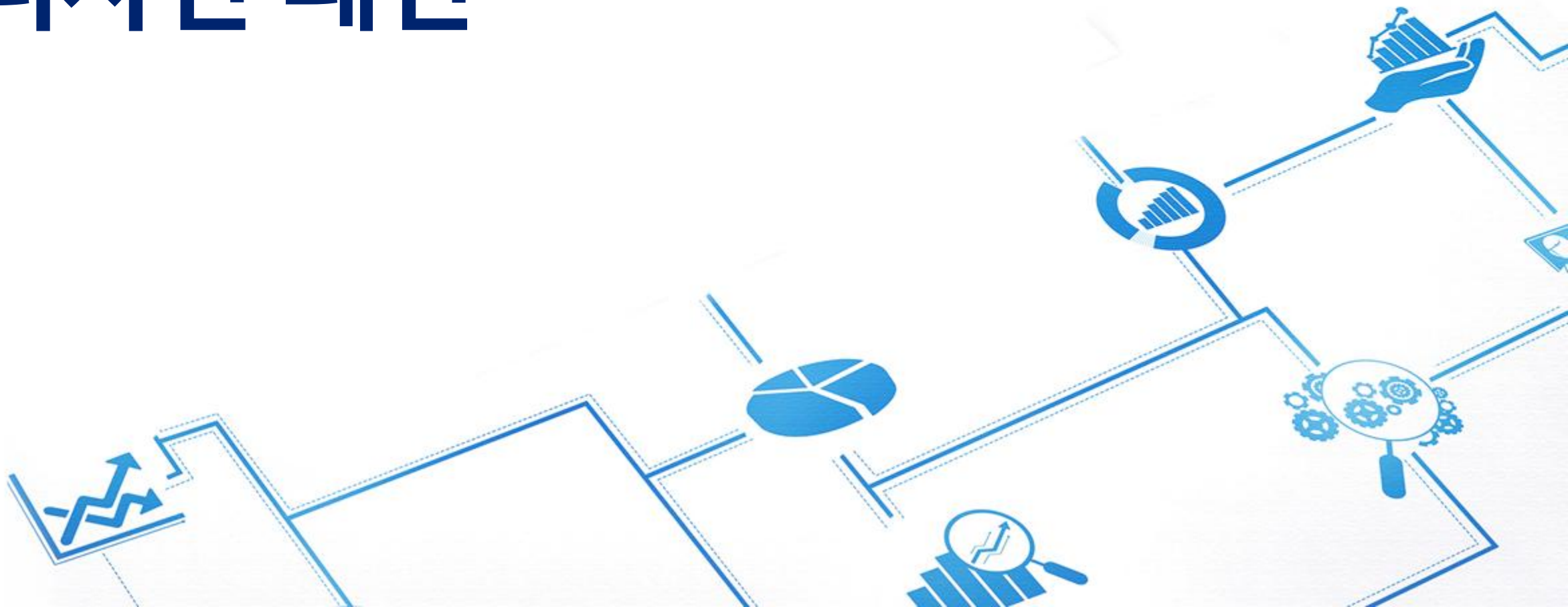


디자인 패턴



학습개요

본 주차의 학습목표와 학습내용을 확인하세요.

학습목표

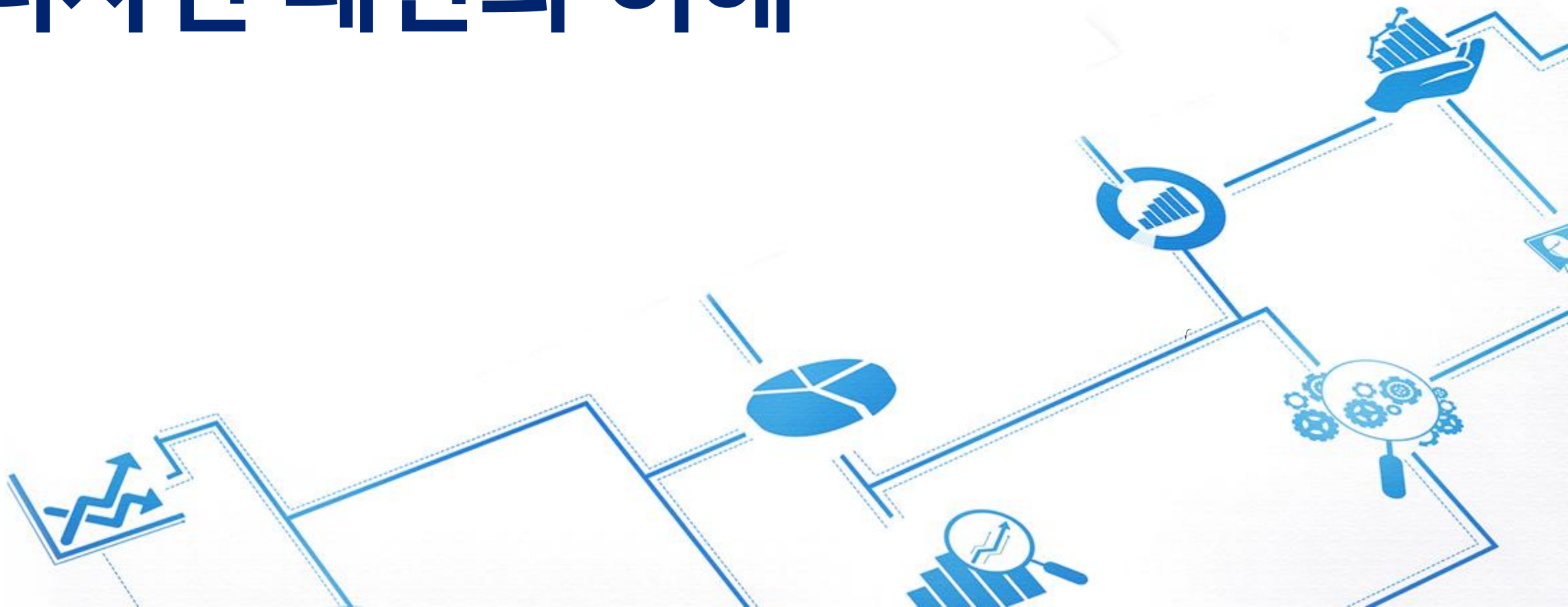
- ☑ 디자인 패턴의 필요성을 알아본다.
- ☑ 행위 패턴을 알아본다.
- ☑ 구조 패턴을 알아본다.
- ☑ 생성 패턴을 알아본다.

학습내용

- 1 디자인 패턴의 이해
- 2 행위 패턴
- 3 구조 패턴
- 4 생성 패턴



디자인 패턴의 이해

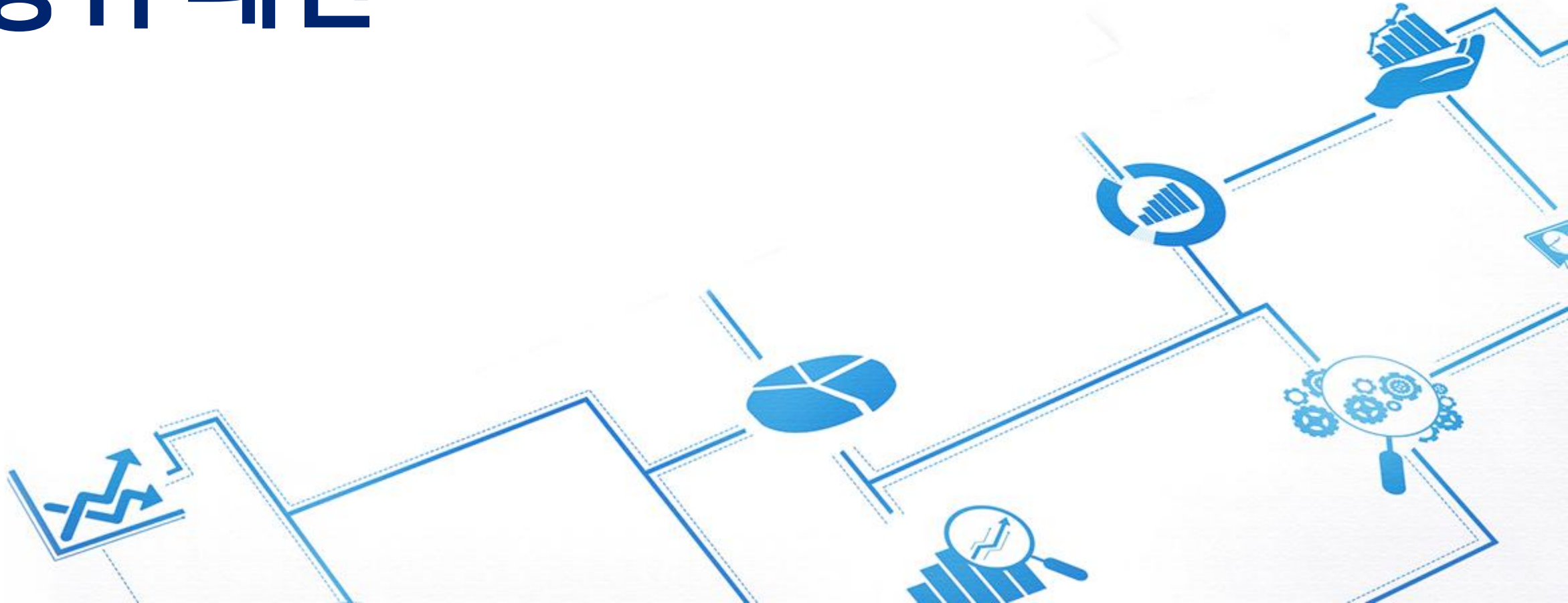


디자인 패턴

- GoF의 디자인 패턴

- 소프트웨어 설계를 위한 지식이나 노하우를 공유할 방법 중 하나
- 설계 중에서 재사용할 경우에 유용한 것을 디자인 패턴으로 정립
- 소프트웨어 설계에 대한 지식이나 노하우가 문제 유형별로 잘 구체화되어 있음
- 동일한 유형의 문제를 해결하는 방법에 대한 지식이나 노하우가 패턴 형태로 충분히 일반화
- 쉽게 재사용할 수 있도록 객체지향 개념에 따른 설계만을 패턴으로 지정

행위 패턴



행위 패턴

- 행위 패턴의 개요

- 반복적으로 사용되는 객체의 상호작용을 패턴화한 것
- 클래스나 객체가 상호작용하는 방법과 책임을 분산하는 방법을 정의
- 메시지 교환과 관련된 것으로 객체 간의 행위나 알고리즘 등과 관련된 패턴

- 행위 패턴의 종류

- strategy 패턴
- template method 패턴
- iterator 패턴
- visitor 패턴
- mediator 패턴
- chain of responsibility 패턴
- state 패턴
- interpreter 패턴
- observer 패턴
- command 패턴
- memento 패턴

strategy 패턴

자주 바뀌는 것이 기능이라면?:
메서드를 클래스로 바꾸고 <<interface>> 타입의 상속 구조를 만들어라

• 예제 프로젝트 이해(포켓몬)

- 이 프로젝트의 특성은 포켓몬의 종류가 자주 추가되고 삭제된다는 점
- 기술이 발전함에 따라 성능 개선을 위해 기능(공격, 패시브)이 수정되기도 하고 새로운 공격 기술이 추가

표 7-1 포켓몬의 종류

포켓몬	기능(공격/패시브)	설명
피카츄	백만 볼트	백만 볼트의 강력한 전압을 상대에게 보내 공격한다.
	스피드	한 번에 빠르게 두 번 공격한다.
푸린	노래 부르기	노래를 불러 상대를 잠재운다.
	회피	30% 확률로 공격을 회피한다.
파이리	불꽃	뜨거운 불꽃을 상대방에게 쏘아 공격한다.
	방어	방어 전략을 사용해 받는 피해를 40% 감소시킨다.

strategy 패턴

- 일반적인 클래스 설계

- 모든 포켓몬이 공통으로 가지는 공격/패시브 기술은 상위 클래스의 `attack()`과 `passive()` 메서드로 놓고 하위 클래스에서 상속

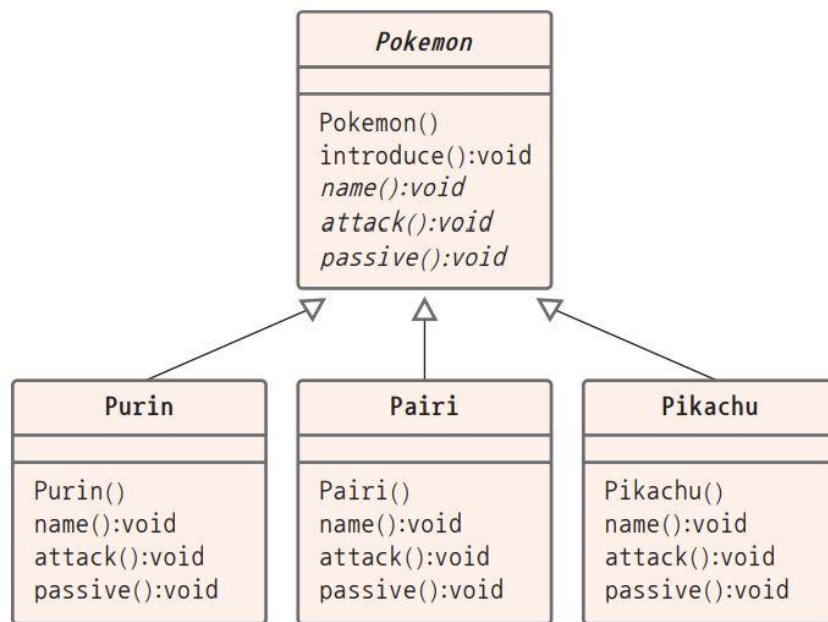


그림 7-1 strategy 패턴 사용 전 클래스 다이어그램

실행 결과

이름: 피카츄, 속성: 번개
공격 스킬: 백만 볼트
패시브 스킬: 스피드

이름: 푸린, 속성: 노멀
공격 스킬: 노래하기
패시브 스킬: 회피

이름: 파이리, 속성: 불
공격 스킬: 불꽃
패시브 스킬: 방어

strategy 패턴

• 일반적인 클래스 설계

코드 7-1	Pokemon 클래스	코드 7-2	Purin 클래스	코드 7-3	Pairi 클래스
	<pre>public abstract class Pokemon { public void introduce() { name(); attack(); passive(); } abstract void name(); abstract void attack(); abstract void passive(); }</pre>	<pre>public class Purin extends Pokemon { public void passive() { System.out.println("패시브 스킬: 회피\n"); // 회피: 30% 확률로 공격 회피 } public void attack() { System.out.println("공격 스킬: 노래하기"); // 노래하기: 노래를 불러 상대를 잠재움 } public void name() { System.out.println("이름: 푸린, 속성: 노멀"); } }</pre>	<pre>public class Pairi extends Pokemon { public void passive() { System.out.println("패시브 스킬: 방어\n"); // 방어: 받는 피해를 40% 감소시킴 } public void attack() { System.out.println("공격 스킬: 불꽃"); // 불꽃: 뜨거운 불꽃을 상대한테 쏴 } void name() { System.out.println("이름: 파이리, 속성: 불"); } }</pre>		
코드 7-4	Pikachu 클래스	코드 7-5	Main 클래스		
	<pre>public class Pikachu extends Pokemon { public void passive() { System.out.println("패시브 스킬: 스피드\n"); // 스피드: 한 번에 두 번 공격함 } public void attack() { System.out.println("공격 스킬: 백만 볼트"); // 백만 볼트: 백만 볼트의 강력한 전압으로 공격함 } void name() { System.out.println("이름: 피카츄, 속성: 번개"); } }</pre>	<pre>public class Main { public static void main(String args[]) { Pokemon pikachu = new Pikachu(); pikachu.introduce(); Pokemon purin = new Purin(); purin.introduce(); Pokemon pairi = new Pairi(); pairi.introduce(); } }</pre>			

strategy 패턴

- 문제: 기능 변화
 - 문제점
 - 기술이 발전함에 따라 공격과 패시브 기술이 계속 교체되지만 이에 대한 해결책이 없음
 - 클래스 설계 원칙 중 OCP(개방 폐쇄 원칙)를 위반하는 것일 뿐 아니라 프로그램을 복잡하게 만듦
 - 해결책
 - 자주 바뀌는 것이 무엇인지 찾아 그들을 별도의 상속 구조로 만들어 사용
 - 자주 바뀌는 것이 attack(), passive()이기 때문에 이들을 클래스로 만들고 상속 구조로 설계
 - 추가되는 공격 기술을 하위 클래스에 계속 추가할 수 있음

strategy 패턴

- strategy 패턴을 적용한 클래스 설계

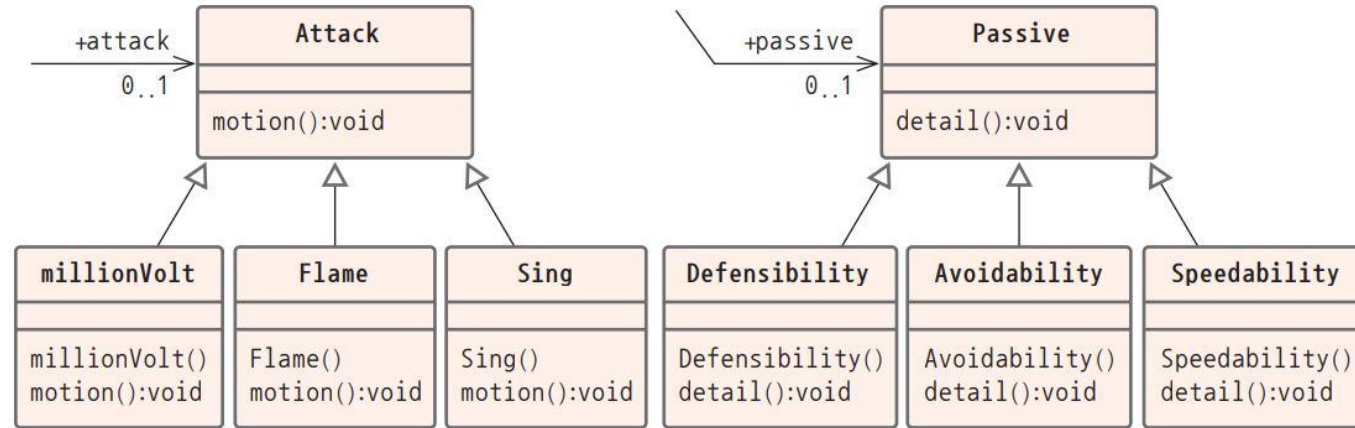


그림 7-2 Attack과 Passive 클래스의 다이어그램 설계

- ① 포켓몬의 종류를 계속 추가/삭제할 수 있도록 Pokemon 클래스를 상위 클래스, 포켓몬 종류를 하위 클래스로 배치한
- ② 상속 구조로 설계
- ③ Pokemon 클래스에서 메서드로 구현한 공격을 별도의 상속 구조로 만들어 사용
- ④ Pokemon 클래스에서 메서드로 구현한 패시브Passive를 별도의 상속 구조로 만들어 사용

strategy 패턴

- strategy 패턴을 적용한 클래스 설계

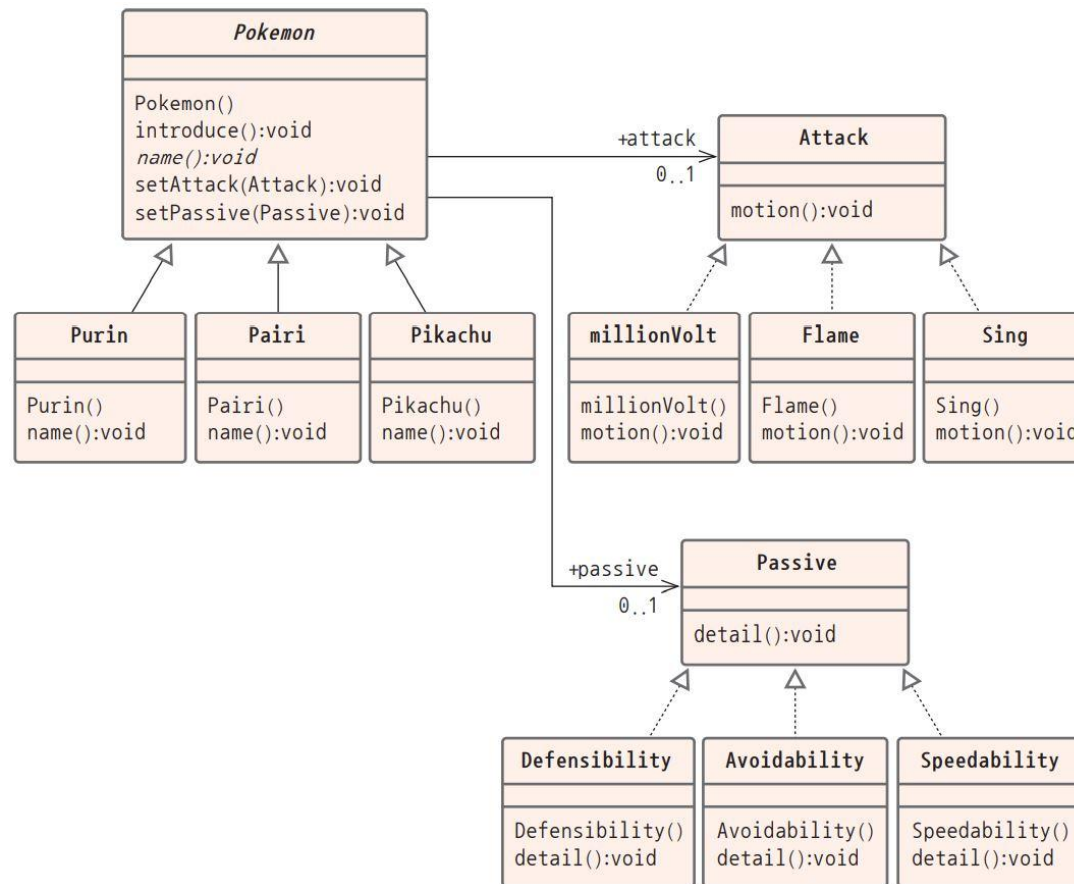


그림 7-3 strategy 패턴을 적용한 클래스 다이어그램

strategy 패턴

- strategy 패턴을 적용한 클래스 설계

코드 7-6	Pokemon 클래스	코드 7-7	Pairi 클래스	코드 7-8	Purin 클래스
	<pre>public abstract class Pokemon { public Attack attack; public Passive passive; public Pokemon() { } public void introduce() { name(); attack.motion(); passive.detail(); } abstract void name(); public void setAttack(Attack attack) { this.attack = attack; } public void setPassive(Passive passive) { this.passive = passive; } }</pre>	<pre>public class Pairi extends Pokemon { public Pairi() { attack = new Flame(); passive = new Defensibility(); } void name() { System.out.println("이름: 파이리, 속성: 불"); } }</pre>		<pre>public class Purin extends Pokemon { public Purin() { attack = new Sing(); passive = new Avoidability(); } void name() { System.out.println("이름: 푸린, 속성: 노멀"); } }</pre>	
코드 7-9	Pikachu 클래스			코드 7-10	Attack 클래스
	<pre>public class Pikachu extends Pokemon { public Pikachu() { attack = new millionVolt(); passive = new Speedability(); } void name() { System.out.println("이름: 피카츄, 속성: 번개"); } }</pre>				<pre>public interface Attack { public void motion(); }</pre>
				코드 7-11	Sing 클래스
					<pre>public class Sing implements Attack { public void motion() { System.out.println("공격 스킬 - 노래하기: 노래를 불러 상대를 잠재움"); } }</pre>

strategy 패턴

- strategy 패턴을 적용한 클래스 설계

코드 7-12

Flame 클래스

```
public class Flame implements Attack {  
    public void motion() {  
        System.out.println("공격 스킬 - 불꽃: 뜨거운 불꽃을 쏘아 공격");  
    }  
}
```

코드 7-14

Passive 클래스

```
public interface Passive {  
    public void detail();  
}
```

코드 7-16

Avoidability 클래스

```
public class Avoidability implements Passive {  
    public void detail() {  
        System.out.println("패시브 스킬 - 회피: 30% 확률로 공격 회피\n");  
    }  
}
```

코드 7-17

Speedability 클래스

```
public class Speedability implements Passive {  
    public void detail() {  
        System.out.println("패시브 스킬 - 스피드: 한 번에 두 번 공격\n");  
    }  
}
```

코드 7-13

millionVolt 클래스

```
public class millionVolt implements Attack {  
    public void motion() {  
        System.out.println("공격 스킬 - 백만 볼트: 백만 볼트의 강력한 전압으로 공격");  
    }  
}
```

코드 7-15

Defensibility 클래스

```
public class Defensibility implements Passive {  
    public void detail() {  
        System.out.println("패시브 스킬 - 방어: 받는 피해 40% 감소시킴\n");  
    }  
}
```

코드 7-18

Main 클래스

```
public class Main {  
    public static void main(String args[]) {  
        Pokemon pikachu = new Pikachu();  
        pikachu.introduce();  
  
        Pokemon pairi = new Pairi();  
        pairi.introduce();  
  
        Pokemon purin = new Purin();  
        purin.introduce();  
  
        purin.setAttack(new millionVolt());  
        purin.setPassive(new Defensibility());  
  
        purin.introduce();  
    }  
}
```

strategy 패턴

- strategy 패턴 정리

- strategy 패턴은 기능(메서드, 전략)이 자주 바뀌기 때문에 이들을 오른쪽에 별도로 구성하고 내부에서 변화하도록 함
- 메서드(전략)마다 각각의 클래스로 먼저 바꾸고 이를 <<interface>> 타입의 상속 구조로 만들어 구현

실행 결과

이름: 피카츄, 속성: 번개

공격 스킬 - 백만 볼트: 백만 볼트의 강력한 전압으로 공격

패시브 스킬 - 스피드: 한 번에 두 번 공격

이름: 파이리, 속성: 불

공격 스킬 - 불꽃: 뜨거운 불꽃을 쏘아 공격

패시브 스킬 - 방어: 받는 피해 40% 감소시킴

이름: 푸린, 속성: 노멀

공격 스킬 - 노래하기: 노래를 불러 상대를 잠재움

패시브 스킬 - 회피: 30% 확률로 공격 회피

이름: 푸린, 속성: 노멀

공격 스킬 - 백만 볼트 : 백만 볼트의 강력한 전압으로 공격

패시브 스킬 - 방어: 받는 피해 40% 감소시킴

state 패턴

자주 바뀌는 것이 상태라면?:
상태를 클래스로 바꾸고 <<interface>> 타입의 상속 구조를 만들어라

- 예제 프로젝트 이해

- 처음의 선풍기는 정지와 송풍 기능만 있는 것
- 정지 상태에서 ON 버튼을 누르면 송풍 상태로 변하고 송풍 상태에서 OFF 버튼을 누르면 정지 상태로 변한
- 선풍기의 바람 상태에 Sleep, 약풍, 중간풍, 강풍이 계속 추가될 것

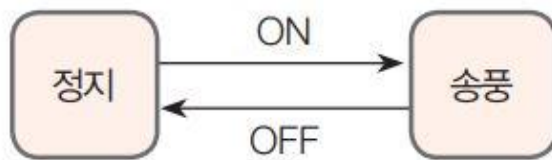


그림 7-4 가장 단순한 선풍기의 상태 변화

state 패턴

- 일반적인 클래스 설계

코드 7-19

ElecFan 클래스

```
public class ElecFan {
    private String State;
    public ElecFan() {
        State = "Stop";
        System.out.println("<<현재 상태: " + State + ">>");
    }
    public void setState(String state) {
        this.State = state;
    }
    public void on_button() {
        if(State == "Stop")
        {
            State = "Wind";
            System.out.println("\n***on 버튼 눌림***\n" + "정지에서 송풍 상태로 바뀜\n");
            System.out.println("<<현재 상태: " + State + ">>");
        }
        else if(State == "Wind") {
            State = "Stop";
            System.out.println("\n***on 버튼 눌림***\n" + "상태 변화 없음\n");
            System.out.println("<<현재 상태: " + State + ">>");
        }
    }
    public void off_button() {
        if(State == "Stop") {
            State = "Stop";
            System.out.println("\n***off 버튼 눌림***\n" + "상태 변화 없음\n");
            System.out.println("<<현재 상태: " + State + ">>");
        }
        else if(State == "Wind") {
            State = "Stop";
            System.out.println("\n***off 버튼 눌림***\n" + "송풍에서 정지 상태로 바뀜\n");
            System.out.println("<<현재 상태: " + State + ">>");
        }
    }
}
```

코드 7-20

Main 클래스

```
public class Main {
    public static void main(String args[]) {
        ElecFan electricfan = new ElecFan();
        electricfan.on_button();
        electricfan.off_button();
    }
}
```

실행 결과

<<현재 상태: Stop>>

on 버튼 눌림

정지에서 송풍 상태로 바뀜

<<현재 상태: Wind>>

off 버튼 눌림

송풍에서 정지 상태로 바뀜

<<현재 상태: Stop>>

state 패턴

- 문제: 상태 변화

- 문제점

- ElecFan 클래스에서는 조건문(if ~ else if)을 사용했는데 이 코드만으로는 상태의 변화를 쉽게 읽을 수 없음
 - 상태(추가, 삭제)가 바뀌면 on_button(), off_button() 메서드에 이를 반영해야 하므로 계속해서 코드 수정이 필요
 - 클래스 설계 원칙 중 OCP(개방 폐쇄 원칙)를 위반하는 것일 뿐 아니라 프로그램을 복잡하게 만들어 유지보수를 어렵게 함

- 해결책

- 자주 바뀌는 것이 상태(송풍, 정지, 수면, 약풍, 중간풍, 강풍 등)이므로 조건문으로 처리하던 상태를 클래스로 변경
 - 클래스를 상속 구조(인터페이스 포함) 형태로 만들어 하위 클래스에서 상태를 추가 및 삭제할 수 있도록 함

state 패턴

- state 패턴을 적용한 클래스 설계

- state 패턴 구조의 설계는 상태의 추가, 삭제 시 ElectricFan 클래스에 영향을 주지 않음
- 상태의 변화는 모두 오른쪽(상태를 클래스로 분리)에서만 이루어짐

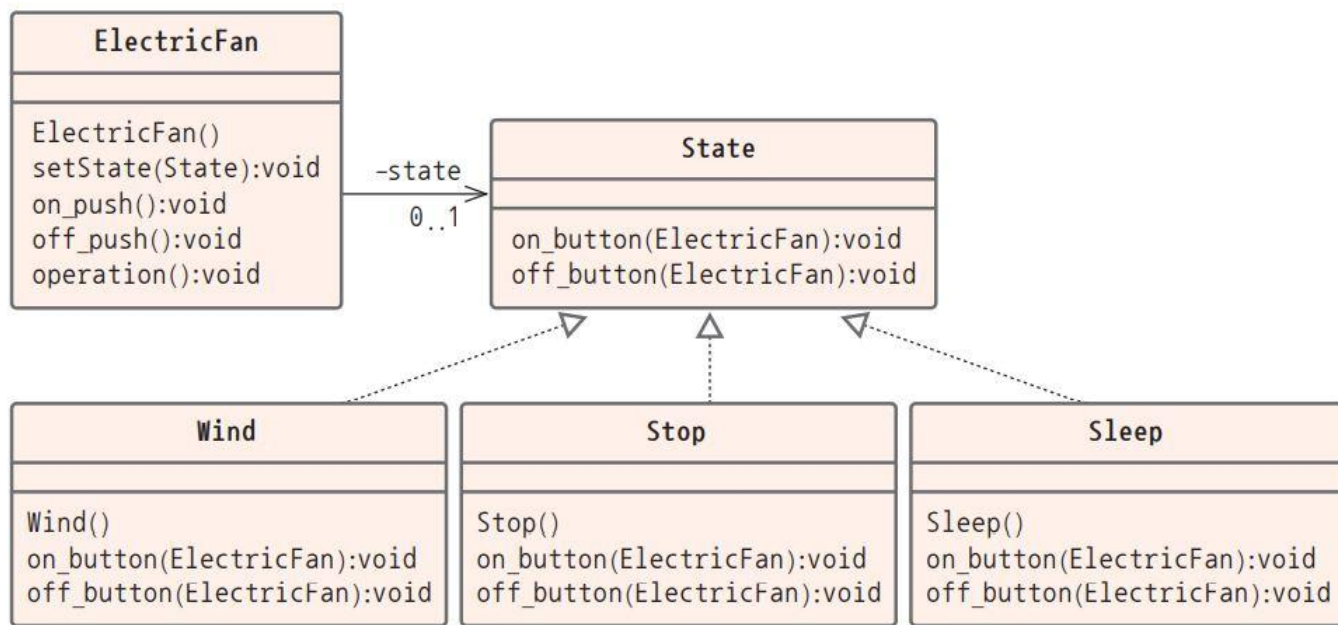


그림 7-5 state 패턴을 적용한 ElectricFan 클래스 다이어그램

state 패턴

- state 패턴을 적용한 클래스 설계

코드 7-21

state 패턴을 적용한 ElectricFan 클래스

```
public class ElectricFan {  
    private State state;  
    public ElectricFan() {  
        state = new Stop();  
    }  
    public void setState(State state) {  
        this.state = state;  
    }  
    public void on_push() {  
        state.on_button(this);  
    }  
    public void off_push() {  
        state.off_button(this);  
    }  
    public void operation() {  
        off_push();  
        on_push();  
        on_push();  
        on_push();  
        off_push();  
    }  
}
```

코드 7-22

State 클래스

```
public interface State {  
    public void on_button(ElectricFan EF);  
    public void off_button(ElectricFan EF);  
}
```

state 패턴

- state 패턴을 적용한 클래스 설계

코드 7-23

state 패턴을 적용한 Stop 클래스

```
public class Stop implements State {
    public Stop() {
        System.out.println("<<현재 상태: 정지>>");
    }
    public void on_button(ElectricFan EF) {
        System.out.println("\n***on 버튼 눌림***\n" + "정지에서 송풍 상태로 바뀜\n");
        EF.setState(new Wind());
    }
    public void off_button(ElectricFan EF) {
        System.out.println("\n***off 버튼 눌림***\n" + "상태 변화 없음\n");
    }
}
```

코드 7-24

state 패턴을 적용한 Wind 클래스

```
public class Wind implements State {
    public Wind() {
        System.out.println("<<현재 상태: 송풍>>");
    }
    public void on_button(ElectricFan EF) {
        System.out.println("\n***on 버튼 눌림***\n" + "송풍에서 수면 상태로 바뀜\n");
        EF.setState(new Sleep());
    }
    public void off_button(ElectricFan EF) {
        System.out.println("\n***off 버튼 눌림***\n" + "송풍에서 정지 상태로 바뀜\n");
        EF.setState(new Stop());
    }
}
```

코드 7-25

state 패턴을 적용한 Sleep 클래스

```
public class Sleep implements State {
    public Sleep() {
        System.out.println("<<현재 상태: 수면>>");
    }
    public void on_button(ElectricFan EF) {
        System.out.println("\n***on 버튼 눌림***\n" + "수면에서 송풍 상태로 바뀜\n");
        EF.setState(new Wind());
    }
    public void off_button(ElectricFan EF) {
        System.out.println("\n***off 버튼 눌림***\n" + "수면에서 정지 상태로 바뀜\n");
        EF.setState(new Stop());
    }
}
```

코드 7-26

Main 클래스

```
public class Main {
    public static void main(String args[]) {
        ElectricFan EF = new ElectricFan();
        EF.operation();
    }
}
```

state 패턴

- state 패턴 정리

- state 패턴은 자주 바뀌는 상태를 클래스로 만들고 이를 <<interface>> 타입의 상속 구조로 만듦
- 상태의 변경이 그 안에서만 이루어지게 하여 주 클래스에 미치는 영향을 최소화하도록 설계한 패턴

실행 결과

<<현재 상태: 정지>>

off 버튼 눌림

상태 변화 없음

on 버튼 눌림

정지 상태에서 송풍 상태로 바뀜

<<현재 상태: 송풍>>

on 버튼 눌림

송풍 상태에서 수면 상태로 바뀜

<<현재 상태: 수면>>

on 버튼 눌림

수면 상태에서 송풍 상태로 바뀜

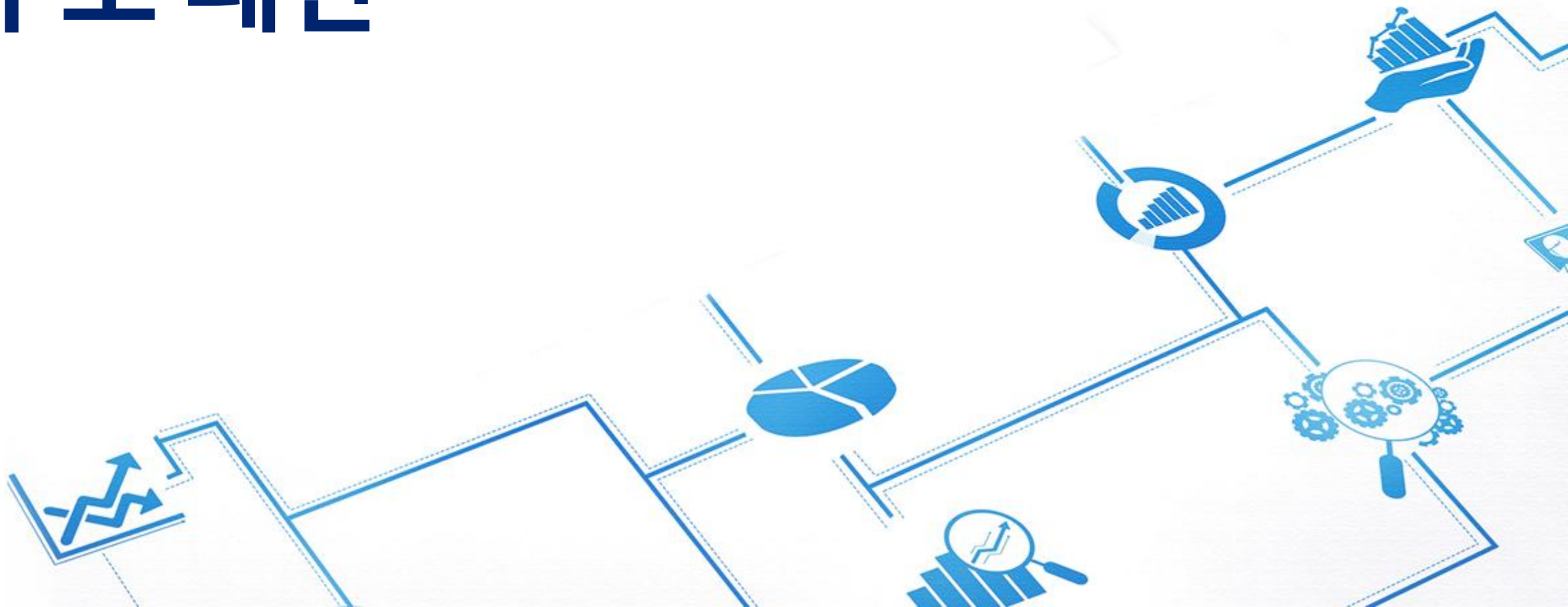
<<현재 상태: 송풍>>

off 버튼 눌림

송풍 상태에서 정지 상태로 바뀜

<<현재 상태: 정지>>

구조 패턴



구조 패턴

- 구조 패턴의 개요
 - 프로그램 내의 데이터나 인터페이스의 구조를 설계하는 데 많이 활용할 수 있음
 - 클래스나 객체의 구성(합성)으로 더 큰 구조를 만들어야 할 때 유용한 디자인 패턴
- 구조 패턴의 종류
 - decorator 패턴
 - adapter 패턴
 - composite 패턴
 - bridge 패턴
 - facade 패턴
 - flyweight 패턴
 - proxy 패턴

decorator 패턴

기본 클래스를 조합해서 많은 클래스를 만들어야 한다면?
많은 클래스를 조용히 처리하라!

- decorator 패턴의 개요
 - 너무 많아질 수 있는 장식을 '조용히(?) 어떻게 처리하는가'를 설명하는 디자인 패턴
 - decorator 패턴은 장식이 주인공으로 여기서의 주인공은 꼭 있어야만 하는 존재가 아니고 장식처럼 없어도 되는 존재
- 예제 프로젝트 이해
 - 토스트를 판매할 때 칼로리를 계산해 알려주는 토스트 가게가 있음
 - 현재 판매하는 메뉴는 치즈 토스트, 야채 토스트, 햄 토스트로 3종류
 - 앞으로 재료도 늘어날 것이고 재료를 혼합한 토스트 종류도 늘어날 예정
 - 기본 재료를 조합해 만드는 토스트의 종류를 하위 클래스로 계속 추가할 수 있는 상속 구조로 만든다면 기본 재료가
 - 추가될 때마다 그것을 조합해 만들어지는 토스트의 종류 (클래스)는 수십 가지로 늘어날 것

decorator 패턴

- decorator 패턴의 적용

- Decorator 패턴을 적용하면 식빵은 상속 구조를 사용
- 빵의 종류가 늘어나는 것에 대해서는 Toast라는 상위 클래스를 만들고 하위 클래스에 식빵의 종류를 둠
- decorator 패턴은 재료를 기본 재료(햄, 치즈, 달걀, 야채)와 복합 재료로 나누는 것부터 시작
- 핵심은 혼합 재료의 숫자만큼 클래스를 만들지 않고 기본 재료만 가지고 해결하는 것
- 기본 재료는 하위 클래스에 놓고 상위 클래스에 ToppiongDecorator라는 이름의 클래스를 만듦
- 혼합 재료는 기본 재료를 계속 늘려 나가는 방식으로 프로그래밍

decorator 패턴

```
Toast toast1 = new NormalBread(); ——— ①  
toast1.addTopping(new Cheese()); ——— ②  
toast1.addTopping(new Ham()); ——— ③  
toast1.server(); ——— ④
```

- ① Toast 타입의 일반식빵인 toast1 객체를 생성
- ② '치즈'를 인자로 갖는 addTopping()을 호출
- ③ '햄'을 인자로 갖는 addTopping()을 호출
- ④ server()를 호출해 지금까지 name에 들어간 이름과 누적된 칼로리를 출력

decorator 패턴

- decorator 패턴의 적용

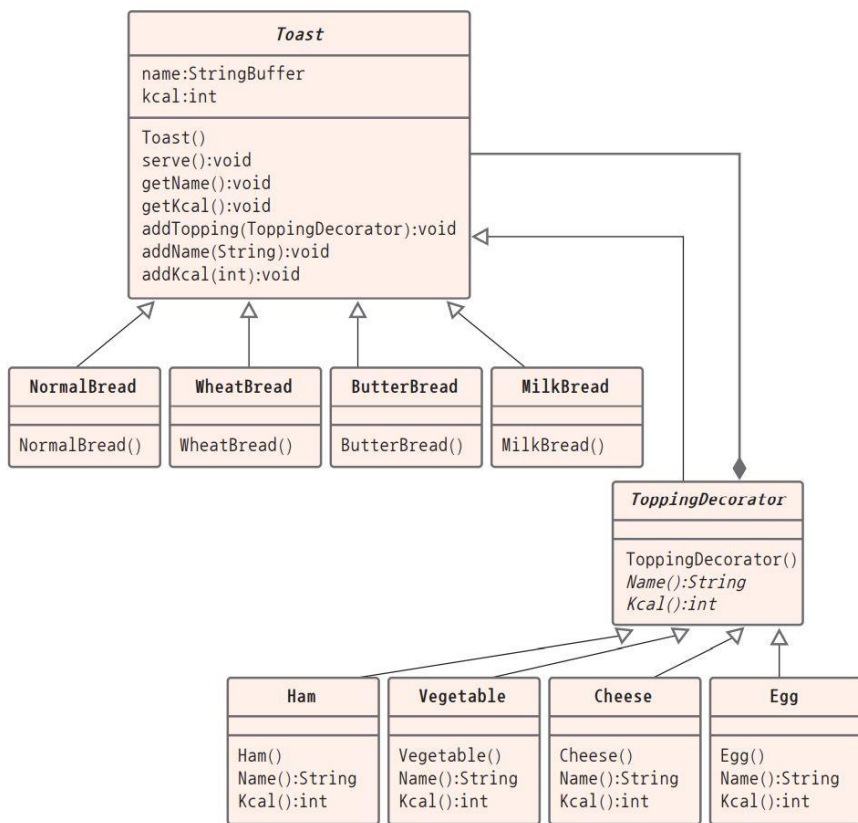


그림 7-6 Decorator 패턴을 적용한 클래스 다이어그램

실행 결과

주문한 토스트 : 햄치즈 식빵 토스트

칼로리 : 550Kcal

주문한 토스트 : 달걀야채 호밀식빵 토스트

칼로리 : 360Kcal

주문한 토스트 : 햄햄햄 버터식빵 토스트

칼로리 : 1000Kcal

decorator 패턴

- decorator 패턴의 적용

코드 7-27

Toast 클래스

```
public abstract class Toast {  
    public StringBuffer name = new StringBuffer();  
    public int kcal;  
    public void serve() {  
        getName();  
        getKcal();  
    }  
    public void getName() {  
        System.out.println("주문한 토스트: " + name);  
    }  
    public void getKcal() {  
        System.out.println("칼로리: " + kcal + "Kcal\n");  
    }  
    public void addTopping(ToppingDecorator td) {  
        addName(td.Name());  
        addKcal(td.Kcal());  
    }  
    public void addName(String name) {  
        this.name.insert(0,name);  
    }  
    public void addKcal(int kcal) {  
        this.kcal += kcal;  
    }  
}
```

코드 7-28

WheatBread 클래스

```
public class WheatBread extends Toast {  
    public WheatBread() {  
        name.append(" 호밀식빵 토스트");  
        addKcal(250);  
    }  
}
```

코드 7-29

NormalBread 클래스

```
public class NormalBread extends Toast {  
    public NormalBread() {  
        name.append(" 식빵 토스트");  
        addKcal(300);  
    }  
}
```

코드 7-30

MilkBread 클래스

```
public class MilkBread extends Toast {  
    public MilkBread() {  
        name.append(" 우유식빵 토스트");  
        addKcal(350);  
    }  
}
```

코드 7-31

ButterBread 클래스

```
public class ButterBread extends Toast {  
    public ButterBread() {  
        name.append(" 버터식빵 토스트");  
        addKcal(400);  
    }  
}
```

코드 7-32

ToppingDecorator 클래스

```
public abstract class ToppingDecorator extends Toast {  
    public abstract String Name();  
    public abstract int Kcal();  
}
```

decorator 패턴

- decorator 패턴의 적용

코드 7-33

Vegetable 클래스

```
public class Vegetable extends ToppingDecorator {  
    @Override  
    public String Name() {  
        return "야채";  
    }  
    @Override  
    public int Kcal() {  
        return 10;  
    }  
}
```

코드 7-34

Cheese 클래스

```
public class Cheese extends ToppingDecorator {  
    public String Name() {  
        return "치즈";  
    }  
    public int Kcal() {  
        return 50;  
    }  
}
```

코드 7-35

Egg 클래스

```
public class Egg extends ToppingDecorator {  
    public String Name() {  
        return "달걀";  
    }  
    public int Kcal() {  
        return 100;  
    }  
}
```

코드 7-37

Main 클래스

```
public class Main {  
    public static void main(String args[]) {  
        Toast toast1 = new NormalBread();  
        toast1.addTopping(new Cheese());  
        toast1.addTopping(new Ham());  
        toast1.serve();  
  
        Toast toast2 = new WheatBread();  
        toast2.addTopping(new Vegetable());  
        toast2.addTopping(new Egg());  
        toast2.serve();  
  
        Toast toast3 = new ButterBread();  
        toast3.addTopping(new Ham());  
        toast3.addTopping(new Ham());  
        toast3.addTopping(new Ham());  
        toast3.serve();  
    }  
}
```

decorator 패턴

- decorator 패턴 정리
 - 객체지향 방법에서는 객체를 안정적으로 추가/삭제할 수 있는 방법으로 상속을 많이 활용
 - 상속 구조는 상위 클래스와 하위 클래스 간에 강한 결합으로 묶이게 되고 객체를 동적으로 확장할 때는 구현하기 쉽지 않음
 - 구성(합성)은 객체를 실행하는 도중에 동적으로 확장한다는 장점

adapter 패턴

- adapter 패턴

구매한 컴포넌트가 맞지 않아 바로 사용할 수 없다면?

adapter를 만들어 사용하라!

- adapter 패턴의 개요

- adapter의 의미: '접속 소켓', '확장 카드', '(물건을 다른 것에) 맞추어 붙이다', '맞추다'
- 호환되지 않는 두 인터페이스를 작동시킬 수 있도록 만든 인터페이스
- adapter 패턴은 다음 두 가지 형태로 사용
 - 클래스 adapter 패턴: 상속을 이용한 어댑터 패턴
 - 인스턴스 adapter 패턴: 위임을 이용한 어댑터 패턴

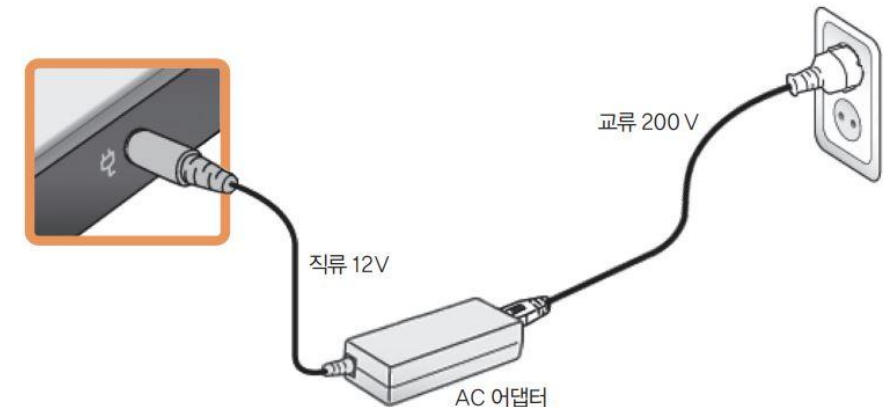


그림 7-7 어댑터의 역할

adapter 패턴

• 예제 프로젝트의 이해

- 삼성 휴대전화에 무선 이어폰 Buds 대신 애플의 무선 이어폰 (Airpods)을 사용할 수 있도록 하는 것
- 두 이어폰의 기능은 재생과 멈춤으로 한정
- Buds는 재생 기능에 play(), 멈춤 기능에 stop() 메서드를 사용
- Airpods은 재생 기능에 playing(), 멈춤 기능에 stopping() 메서드를 사용

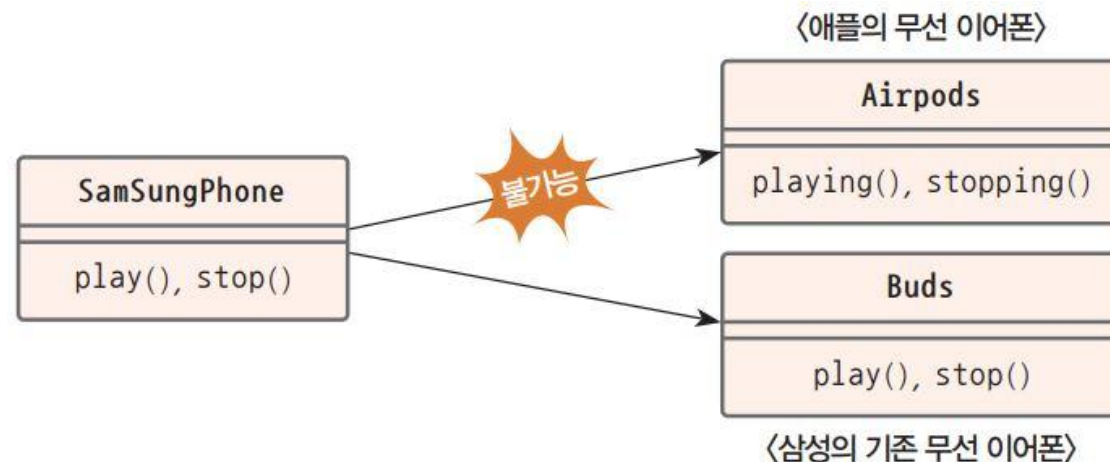


그림 7-8 삼성 휴대전화에서 애플 무선 이어폰 사용

adapter 패턴

- adapter 패턴의 적용
 - <<interface>>타입의 AirPodsInterface 클래스를 정의하고 그 메서드는 Buds에서 사용하는 play(), stop() 메서드와 같게 함
 - 어댑터로 AirPodsAdapter 클래스를 만들
 - AirPods 클래스로부터 상속을 받지만 AirPodsInterface 클래스로 인터페이스 구현으로 이루어졌기 때문에 자바에서 가능

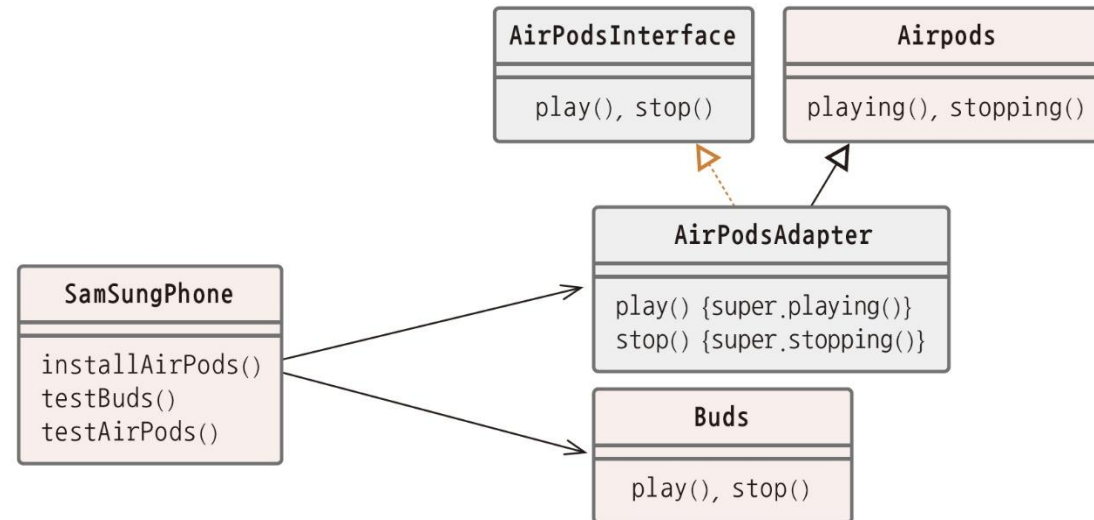


그림 7-9 어댑터로 AirPods 사용하기

adapter 패턴

- adapter 패턴의 적용

코드 7-38

AirPods 클래스

```
public class AirPods {  
    public AirPods() {  
        System.out.println("\n새로 구입한 Apple AirPods입니다.");  
    }  
    public void playing() {  
        System.out.println("Apple AirPods 음악 재생 중....");  
    }  
    public void stopping() {  
        System.out.println("Apple AirPods 음악을 중지합니다!");  
    }  
}
```

코드 7-40

AirPodsAdapte 클래스

```
class AirPodsAdapter extends AirPods implements AirPodsInterface {  
    public AirPodsAdapter() {  
        System.out.println("AirPods이 SamsungPhone과 호환됩니다.");  
    }  
    public void play() {  
        super.playing();  
    }  
    public void stop() {  
        super.stopping();  
    }  
}
```

코드 7-39

Buds 클래스

```
public class Buds {  
    public Buds() {  
        System.out.println("무선 이어폰 호환 시스템.");  
    }  
    public void play() { // 음악을 재생하는 표준 API  
        System.out.println("삼성 Buds 음악 재생 중...");  
    }  
    public void stop() { // 음악을 정지하는 표준 API  
        System.out.println("삼성 Buds 음악을 중지합니다!");  
    }  
}
```

코드 7-41

AirPodsInterface 클래스

```
public interface AirPodsInterface {  
    void play();  
    void stop();  
}
```

adapter 패턴

- adapter 패턴의 적용

코드 7-42

SamsungPhone 클래스

```
public class SamsungPhone {
    Buds buds = new Buds();
    AirPodsAdapter airpodsadapter;
    public SamsungPhone() {
        installAirPods();
        System.out.println();
        testBuds();
        System.out.println();
        testAirPods();
    }
    public void installAirPods() {
        airpodsadapter = new AirPodsAdapter();
        System.out.println("Buds와 새로 구입한 AirPods을 연결합니다.");
    }
    public void testBuds() {
        buds.play();
        buds.stop();
    }
    public void testAirPods() {
        airpodsadapter.play();
        airpodsadapter.stop();
    }
}
```

코드 7-43

main 클래스

```
public class main {
    public static void main(String[] args) {
        SamsungPhone samsungphone = new SamsungPhone();
    }
}
```

adapter 패턴

- adapter 패턴의 적용

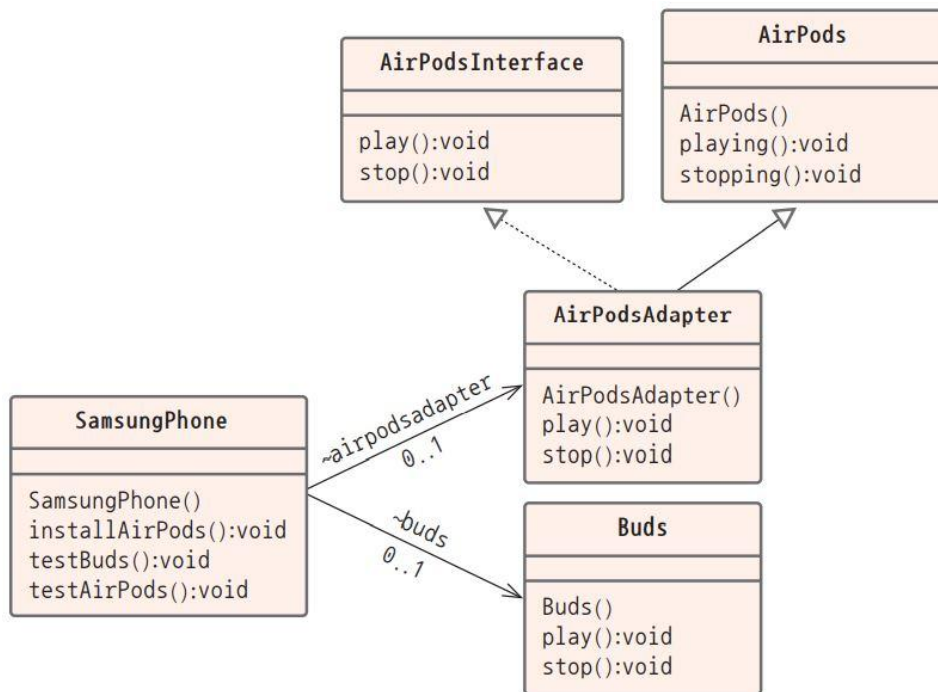


그림 7-10 adapter 패턴을 적용한 클래스 다이어그램

실행 결과

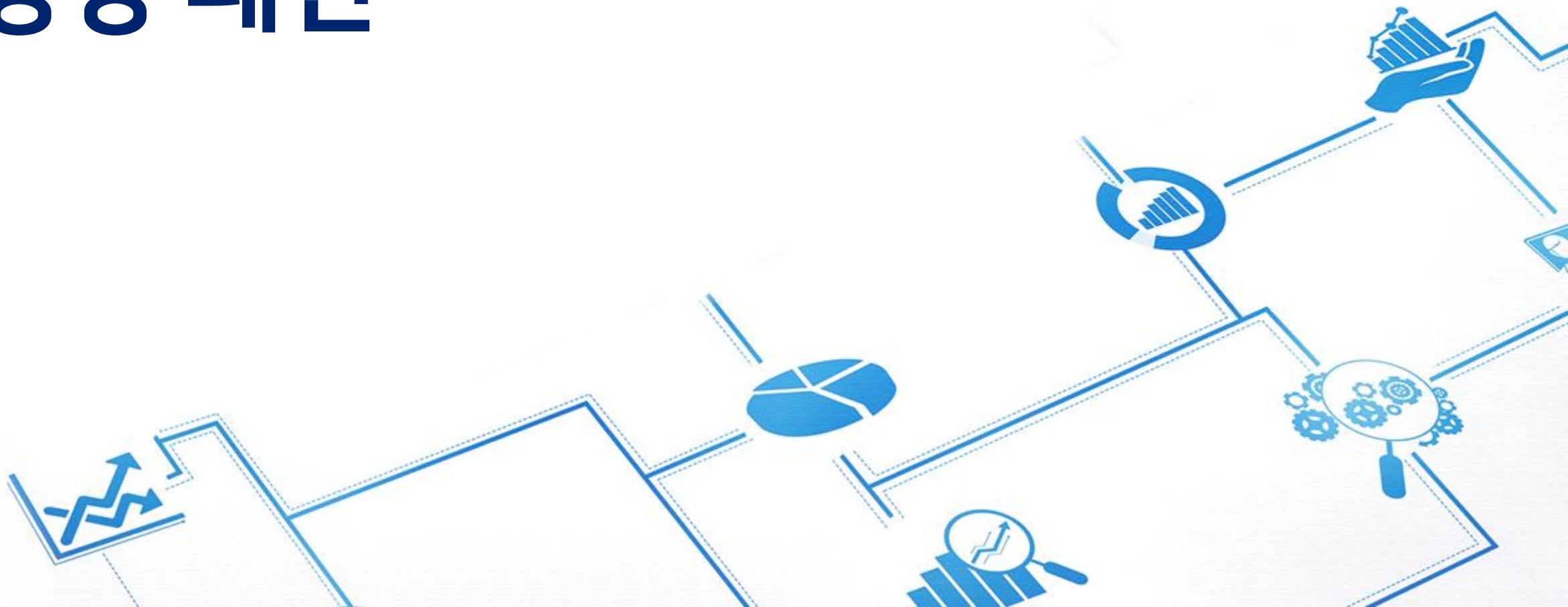
무선 이어폰 호환 시스템

새로 구입한 Apple AirPods입니다.
AirPods이 SamsungPhone과 호환됩니다.
Buds와 새로 구입한 AirPods을 연결합니다.

삼성 Buds 음악 재생 중...
삼성 Buds 음악을 중지합니다!

Apple AirPods 음악 재생 중...
Apple AirPods 음악을 중지합니다!

생성 패턴



생성 패턴

- 생성 패턴의 개요

- 객체의 생성과 참조 과정을 추상화해 특정 객체의 생성 과정을 분리
- 누가 언제 변경하더라도 전체 시스템에 미치는 영향을 최소화하도록 만들어줌
- 코드의 유연성을 높일 수 있고 유지관리를 쉽게 만듦

- 생성 패턴의 종류

- factory method 패턴
- singleton 패턴
- prototype 패턴
- builder 패턴
- abstract factory 패턴

factory method 패턴

객체 생성을 직접 하지 않고 누군가에게 맡기고 싶다면?
factory method 패턴을 생각해보라

- factory method 패턴의 개요

- 객체지향 언어를 사용하면 클래스에서 다른 클래스의 새로운 객체를 만들 때 두 클래스 사이에는 의존 관계가 발생
- 서로 간에 강한 결합을 이루게 되고 변경에 따른 영향을 받아 코드의 유연한 확장이나 변경 같은 유지보수를 어렵게 만듦
- 필요한 객체를 클래스에서 생성하지 말고 객체 생성만 전문으로 하는 클래스를 만들어 객체 생성을 모두 생성해 해결

factory method 패턴

- factory method 패턴의 단계
 - 1단계: 객체 생성을 담당하는 factory 클래스를 만들어 그곳에서 객체를 생성
 - 2단계: 객체를 생성할 수 있는 메서드를 만들어 그 메서드가 factory 클래스 역할을 하도록 함, factory 클래스는 삭제
- 예제 프로젝트 이해
 - 게임 서버는 게임의 종류가 늘어날 때마다 추가
 - 하나의 게임을 선택하면 게임 서버는 '정상 연결' 메시지와 함께 게임명, 게임 버전, '게임 실행 준비 완료' 메시지를 화면에 띄움
 - 추후에는 게임 서버도 국가별로 별도로 둘 예정이며 게임도 그 나라의 특성에 맞는 게임으로 수정할 예정

factory method 패턴

- 일반적인 클래스 설계
 - factory method 패턴을 사용하지 않는 경우

코드 7-44

factory method 패턴을 사용하지 않은 GameServer 클래스

```
public class GameServer {
    public SuperMario supermario = new SuperMario();
    public Tetris tetris = new Tetris();
    public String title;
    public String version;
    public GameServer() {
        System.out.println("GameServer - 정상 연결 완료\n");
    }
    public void Game_Play(String game) {
        choiceGame(game);
        Start();
    }
    public void Start() {
        BootingGame();
        RunGame();
    }
    public void BootingGame() {
        System.out.println("게임명: " + title);
        System.out.println("게임 버전: " + version);
        System.out.println(title + " 게임이 실행 준비되었습니다\n");
    }
    public void RunGame() {
        System.out.println(title + "을 시작합니다.\n");
    }
    public void choiceGame(String game) {
        if (game.equals("supermario")) {
            title = supermario.returnTitle();
            version = supermario.returnVersion();
        } else if (game.equals("tetris")) {
            title = tetris.returnTitle();
            version = tetris.returnVersion();
        } else
            System.out.println("지원하지 않는 게임입니다.");
    }
}
```

factory method 패턴

- 일반적인 클래스 설계

- 게임이 계속 추가되면 그때마다 GameServer 클래스를 수정해야함
- 클래스 설계 원칙 중 OCP(개방 폐쇄 원칙) 위반이고 디자인 패턴의 취지와 맞지 않음
- 객체 내부에서 다른 객체를 생성하면 객체 사이에 의존 관계가 발생

```
public SuperMario supermario = new SuperMario();  
public Tetris tetris = new Tetris();
```

- new는 컴파일 과정에서 객체를 생성하고 이를 메모리에 할당하는 역할
- 위에서는 supermario 객체를 생성하고 tetris 객체를 생성
- 다른 게임이 추가되면 마찬가지로 new를 사용해 객체를 생성할 것

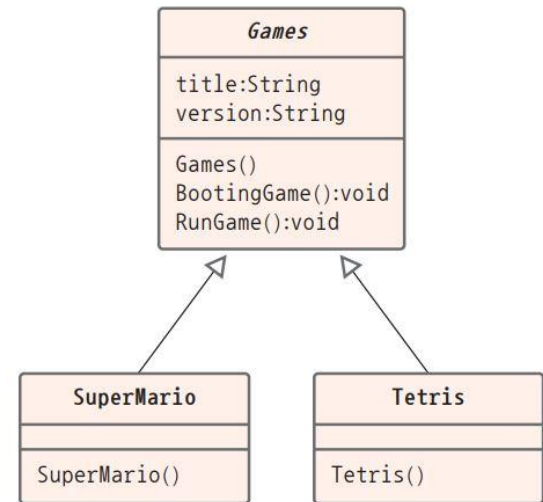


그림 7-11 게임의 상속 구조

factory method 패턴

- factory method 패턴을 적용한 클래스 설계 - 1단계

코드 7-45

1단계: factory method 패턴을 적용한 GameServer 클래스

```
public class GameServer {  
    public GameServerFactory gsf;  
    public GameServer() {  
        System.out.println("GameServer-정상 연결 완료\n");  
    }  
    public GameServer(GameServerFactory gsf) {  
        this.gsf = gsf;  
    }  
    public void execute(String game) {  
        Games games;  
        games = gsf.chooseGame(game);  
        games.BootGame();  
        games.RunGame();  
    }  
}
```

코드 7-46

GameServerFactory 클래스

```
public class GameServerFactory {  
    SuperMario supermario = new SuperMario();  
    Tetris tetris = new Tetris();  
    public Games chooseGame (String game) {  
        if (game.equals("supermario")) {  
            return supermario;  
        }  
        else if (game.equals("tetris")) {  
            return tetris;  
        }  
        else {  
            System.out.println("지원하지 않는 게임입니다.");  
            return null;  
        }  
    }  
}
```

factory method 패턴

- factory method 패턴을 적용한 클래스 설계 - 1단계

코드 7-47

Game 클래스

```
public abstract class Games {  
    public String title, version;  
    public void BootingGame() {  
        System.out.println("게임명: " + title);  
        System.out.println("게임 버전: " + version);  
        System.out.println(title + " 게임이 실행 준비되었습니다\n");  
    }  
    public void RunGame() {  
        System.out.println(title + "을 시작합니다.\n");  
    }  
}
```

코드 7-48

SuperMario 클래스

```
public class SuperMario extends Games {  
    public SuperMario() {  
        title = "Super Mario";  
        version = "v1.0";  
    }  
}
```

코드 7-49

Tetris 클래스

```
public class Tetris extends Games {  
    public Tetris() {  
        title = "Tetris";  
        version = "v1.3";  
    }  
}
```

코드 7-49

Main 클래스

```
public class Main {  
    public static void main(String args[]) {  
        GameServer gs = new GameServer(new GameServerFactory());  
        gs.execute("supermario");  
        gs.execute("tetris");  
    }  
}
```

실행 결과

GameServer-정상 연결 완료

게임명: Super Mario

게임 버전: v1.0

Super Mario 게임이 실행 준비되었습니다

Super Mario를 시작합니다.

게임명: Tetris

게임 버전: v1.3

Tetris 게임이 실행 준비되었습니다

Tetris를 시작합니다.

factory method 패턴

- factory method 패턴을 적용한 클래스 설계 - 1단계
 - 게임의 종류는 Games라는 추상 클래스의 하위 클래스에 위치
 - 새로운 게임 하나가 추가되면 GameServerFactory 클래스에서 객체를 생성하고 if 문에 추가
 - 게임의 추가/삭제에 따른 변화가 GameServer에 영향을 미치지 않음

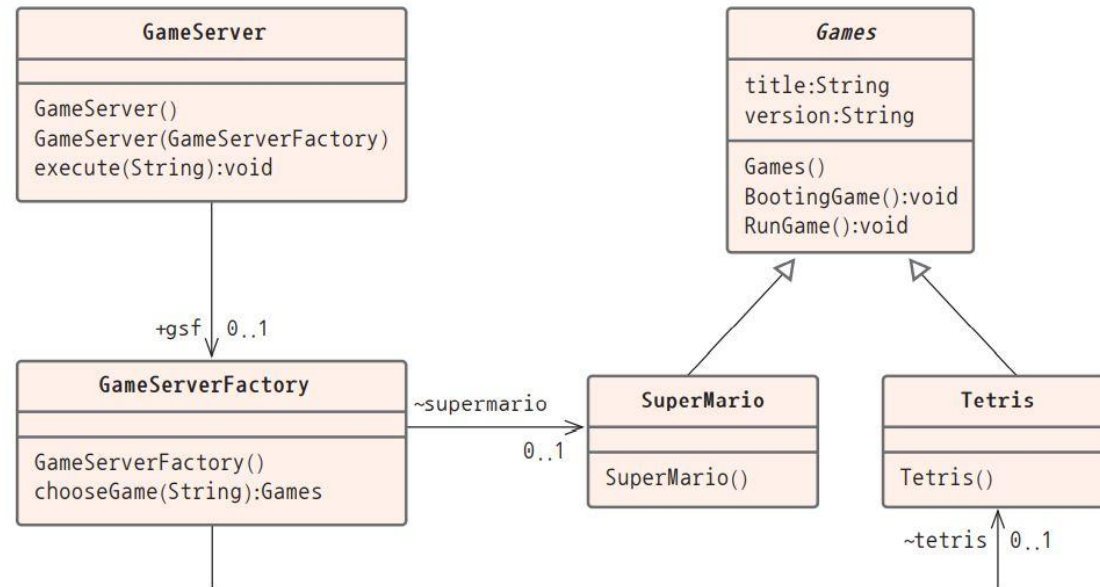


그림 7-12 factory method 패턴을 적용한 1차 클래스 다이어그램

factory method 패턴

- factory method 패턴을 적용한 클래스 설계 - 2단계
 - GameServer를 계속 추가할 수 있도록 KRGameServer와 JPGameServer로 GameServer를 확장
 - 국가별 특성에 맞게 약간 수정한 KRSuperMario, KRTetris, JPSuperMario, JPTetris를 추가해 Games를 확장
- 1단계와의 차이점
 - ① GameServer 확장을 고려한 상속 구조
 - ② 나라별 game 증가를 고려한 상속 구조
 - ③ GameServerFactory 클래스가 사라짐

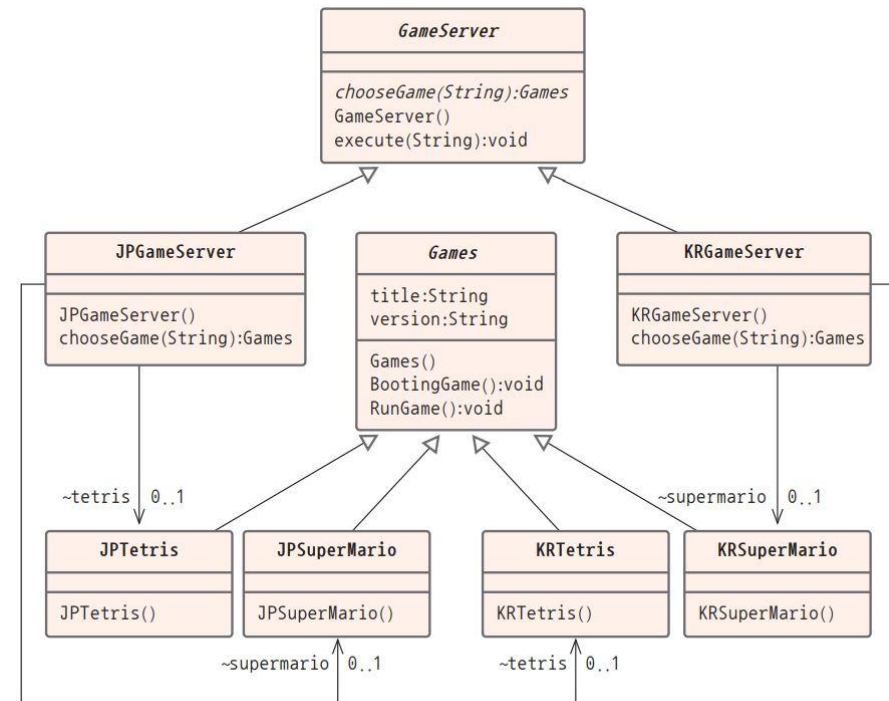


그림 7-13 factory method 패턴을 적용한 최종 클래스 다이어그램

factory method 패턴

- factory method 패턴을 적용한 클래스 설계 - 2단계

코드 7-51

GameServer 클래스

```
public abstract class GameServer {
    protected abstract Games chooseGame(String game);
    public GameServer() {
        System.out.println("GameServer-정상 연결 완료\n");
    }
    public void execute(String game) {
        Games games;
        games = chooseGame(game);
        games.BootigGame();
        games.RunGame();
    }
}
```

코드 7-52

KRGameServer 클래스

```
public class KRGameServer extends GameServer {
    KRSuperMario supermario = new KRSuperMario();
    KRTetris tetris = new KRTetris();
    public Games chooseGame (String game) {
        if (game.equals("supermario")) {
            return supermario;
        }
        else if (game.equals("tetris")) {
            return tetris;
        }
        else {
            System.out.println("지원하지 않는 게임입니다.");
            return null;
        }
    }
}
```

코드 7-53

Games 클래스

```
public abstract class Games {
    public String title, version;
    public void BootigGame() {
        System.out.println("게임명: " + title);
        System.out.println("게임 버전: " + version);
        System.out.println(title + " 게임이 실행 준비되었습니다\n");
    }
    public void RunGame() {
        System.out.println(title + "을 시작합니다.\n");
    }
}
```

코드 7-54

KRTetris 클래스

```
public class KRTetris extends Games {
    public KRTetris() {
        title = "Tetris_Korean";
        version = "v1.3";
    }
}
```

- KRSuperMario, JPSuperMario, JPTetris 클래스의 코드는 KRTetris 클래스의 코드와 유사

factory method 패턴

- factory method 패턴을 적용한 클래스 설계 - 2단계

코드 7-55

Main 클래스

```
public class Main {  
    public static void main(String args[]) {  
        GameServer gs = new KGameServer();  
        gs.execute("supermario");  
        gs = new JGameServer();  
        gs.execute("tetris");  
    }  
}
```

실행 결과

GameServer-정상 연결 완료

게임명: Super Mario_Korean

게임 버전: v1.0

Super Mario_Korean 게임이 실행 준비되었습니다.

Super Mario_Korean을 시작합니다.

GameServer-정상 연결 완료

게임명: Tetris_Japanese

게임 버전: v1.12

Tetris_Japanese 게임이 실행 준비되었습니다.

Tetris_Japanese를 시작합니다.

factory method 패턴

- factory method 패턴 정리
 - 1단계: new를 사용하지 않고 객체를 생성하려면 객체를 생성할 수 있는 클래스를 만들어 모든 객체를 생성하도록 위임
 - 2단계: factory 클래스의 역할(객체 생성)을 하나의 메서드를 상위 클래스에 만들어 그 메서드가 하도록 하고 이 메서드의 구현은 하위 클래스에게 위임

singleton 패턴

객체를 오직 1개만 만들어야 한다면?

singleton 패턴을 생각해보라!

- singleton 패턴의 개요
 - 의도적으로 단 하나의 객체만 생성하도록 하는 경우 사용
 - 객체가 프로그램 내에서 오직 하나만 생성되도록 보장
 - 객체를 하나만 생성해 사용하면 유용한 곳으로는 스레드 풀, 캐시, 로그 기록용 객체 등이 있음
- 일반적으로 작성하던 코드와 다른 점
 - 생성자가 private으로 선언
 - 객체를 생성할 때 static으로 선언

singleton 패턴

코드 7-56

singleton 패턴을 사용하지 않는 경우

```
public class NonSingleton {  
    public NonSingleton() {  
        System.out.println("Singleton 패턴을 사용하지 않는 경우");  
    }  
}
```

코드 7-57

singleton 패턴을 사용하는 경우

```
public class Singleton {  
    private static Singleton singleton = new Singleton();  
    private Singleton() {  
        System.out.println("Singleton 패턴을 사용하는 경우");  
    }  
    public static Singleton getInstance() {  
        return singleton;  
    }  
}
```

실행 결과

```
Singleton 패턴을 사용하지 않는 경우  
Singleton 패턴을 사용하지 않는 경우
```

코드 7-58

Main 클래스

```
public class Main {  
    public static void main(String[] args) {  
        NonSingleton nonSingleton_01 = new NonSingleton();  
        NonSingleton nonSingleton_02 = new NonSingleton();  
        Singleton singleton_01 = Singleton.getInstance();  
        Singleton singleton_02 = Singleton.getInstance();  
    }  
}
```

실행 결과

```
Singleton 패턴을 사용하는 경우
```

- 예제 프로젝트 이해
 - 커피머신 예제를 이용해 singleton 패턴을 설명

singleton 패턴

- singleton 패턴을 사용하지 않은 일반 프로그램
 - 실행을 해보면 객체가 3개 만들어지고 예상한 결과를 얻을 수 있지만 이는 운영체제의 병행 제어 장치의 영향 때문

코드 7-59

EspressoMachine 클래스

```
public class EspressoMachine {  
    public void Brew(String Name, String Beverage) {  
        System.out.println(Name + "님이 " + Beverage + "를 내립니다.");  
        System.out.println(Name+"님의 "+Beverage+"가 완성되었습니다.");  
    }  
}
```

코드 7-60

User 클래스

```
public class User {  
    public EspressoMachine EM = new EspressoMachine();  
    public String Name;  
    public String Beverage;  
    public User(String Name, String Beverage) {  
        this.Name = Name;  
        this.Beverage = Beverage;  
    }  
    public void make() {  
        EM.Brew(Name, Beverage);  
    }  
}
```

코드 7-61

Main 클래스

```
public class Main {  
    public static void main(String args[]) {  
        User user1 = new User("이장훈", "스팀밀크");  
        User user2 = new User("이여진", "바닐라라떼");  
        User user3 = new User("김치수", "아메리카노");  
        user1.make();  
        user2.make();  
        user3.make();  
    }  
}
```

실행 결과

이장훈님이 스팀밀크를 내립니다.
이장훈님의 스팀밀크가 완성되었습니다.
이여진님이 바닐라라떼를 내립니다.
이여진님의 바닐라라떼가 완성되었습니다.
김치수님이 아메리카노를 내립니다.
김치수님의 아메리카노가 완성되었습니다.

singleton 패턴

- singleton 패턴을 사용하지 않은 일반 프로그램 - Thread 사용
 - 3명의 사용자가 원하는 음료를 내리고 완성된 것이 섞여 있고 결과가 다름
 - 목적은 객체가 오직 1개만 생성되고 다른 사용자가 그것을 참조하도록 하는 것이므로 문제가 있음

코드 7-62

EspressoMachine 클래스

```
public class EspressoMachine {
    public void Brew(String Name, String Beverage) {
        try {
            System.out.println(Name + "님의 "+Beverage+"를 내립니다.");
            Thread.sleep(100);
            System.out.println(Name+ "님의 "+Beverage+"가 완성되었습니다.");
        }
        catch(Exception e) {
            e.printStackTrace();
        }
    }
}
```

코드 7-63

User 클래스

```
public class User extends Thread {
    // [코드 7-60]의 User 클래스와 동일
}
```

코드 7-64

Main 클래스

```
public class Main {
    public static void main(String args[]) {
        User user1 = new User("이장훈","스팀밀크");
        User user2 = new User("이여진","바닐라라떼");
        User user3 = new User("김치수","아메리카노");
        user1.start();
        user2.start();
        user3.start();
    }
}
```

실행 결과

이장훈님이 스팀밀크를 내립니다.
이여진님이 바닐라라떼를 내립니다.
김치수님이 아메리카노를 내립니다.
이장훈님의 스팀밀크가 완성되었습니다.
김치수님의 아메리카노가 완성되었습니다.
이여진님의 바닐라라떼가 완성되었습니다.

실행 결과

이장훈님의 스팀밀크를 내립니다.
이여진님의 바닐라라떼를 내립니다.
김치수님의 아메리카노를 내립니다.
이장훈님의 스팀밀크가 완성되었습니다.
이여진님의 바닐라라떼가 완성되었습니다.
김치수님의 아메리카노가 완성되었습니다.

실행 결과

이여진님의 바닐라라떼를 내립니다.
이장훈님의 스팀밀크를 내립니다.
김치수님의 아메리카노를 내립니다.
이여진님의 바닐라라떼가 완성되었습니다.
이장훈님의 스팀밀크가 완성되었습니다.
김치수님의 아메리카노가 완성되었습니다.

실행 결과

이장훈님의 스팀밀크를 내립니다.
김치수님의 아메리카노를 내립니다.
이여진님의 바닐라라떼를 내립니다.
이여진님의 바닐라라떼가 완성되었습니다.
김치수님의 아메리카노가 완성되었습니다.
이장훈님의 스팀밀크가 완성되었습니다.

singleton 패턴

- singleton 패턴을 사용한 프로그램 - Thread 사용, 동기화 비사용
 - ① 생성자가 객체를 한 번만 생성하고 외부로부터 생성자를 호출할 수 없게 만듦
 - EspressoMachine 클래스에서 생성자의 접근 제어자를 private으로 선언
 - ② 클래스 타입을 static(정적), private으로 선언
 - 자신의 클래스 타입을 static(정적)으로 선언하고 외부에서 값을 변경하지 못하도록 private으로 선언
 - ③ getInstance()를 사용
 - getInstance()를 만들어 사용하도록 함
 - 외부에서는 EspressoMachine 클래스의 객체를 얻기 위해 getInstance()를 호출해 사용할 수밖에 없음

singleton 패턴

- singleton 패턴을 사용한 프로그램 - Thread 사용, 동기화 비사용

코드 7-65

EspressoMachine 클래스

```
public class EspressoMachine {
    public static EspressoMachine EM;
    private EspressoMachine() {
        System.out.println("에스프레소 머신 생성");
    }
    private String[] vanillaLatte = {"에스프레소", "우유", "바닐라시럽"};
    private String[] americano = {"에스프레소", "물"};
    private String[] stimilk = {"우유", "시럽"};
    private String condiments = "";
    public static EspressoMachine getInstance() {
        if(EM == null) {
            EM = new EspressoMachine();
        }
        return EM;
    }
    public void Brew(String Name, String Beverage) {
        System.out.println(Name + "님이 " + Beverage + "를 내립니다.");
        Blend(Beverage);
        System.out.println(Name + "님의 "+Beverage+"가 완성되었습니다. (첨가물 : "+condiments+"");
        condiments = "";
    }
    public void Blend(String Beverage) {
        if (Beverage.equals("바닐라라떼")) {
            for (int i = 0; i < 3; i++) {
                System.out.println(vanillaLatte[i] + "를 추가.");
                try {
                    Thread.sleep(1000);
                }
                catch(Exception e) {
                    e.printStackTrace();
                }
            }
            condiments = new StringBuffer(condiments).append(vanillaLatte[i]).append(", ").toString();
        }
    }
}
```

=====

singleton 패턴

- singleton 패턴을 사용한 프로그램 - Thread 사용, 동기화 비사용
 - 스레드를 사용해 실행 결과를 보니 결과가 섞여서 출력

코드 7-66

User 클래스

```
public class User extends Thread {
    public EspressoMachine EM;
    public String Name;
    public String Beverage;
    public User(String Name, String Beverage) {
        this.Name = Name;
        this.Beverage = Beverage;
    }
    public void run() {
        EM = EspressoMachine.getInstance();
        EM.Brew(Name, Beverage);
    }
}
```

코드 7-67

Main 클래스

```
public class Main {
    public static void main(String args[]) {
        User user1 = new User("이장훈", "스팀밀크");
        user1.start();
        User user2 = new User("이여진", "바닐라라떼");
        user2.start();
        User user3 = new User("김치수", "아메리카노");
        user3.start();
    }
}
```

실행 결과

에스프레소 머신 생성
에스프레소 머신 생성
김치수님이 아메리카노를 내립니다.
에스프레소를 추가.
에스프레소 머신 생성
이장훈님이 스팀밀크를 내립니다.
이여진님이 바닐라라떼를 내립니다.
우유를 추가.
에스프레소를 추가.
바닐라시럽을 추가.
우유를 추가.
물을 추가.
시럽을 추가.
김치수님의 아메리카노가 완성되었습니다. (첨가물 : 에스프레소, 물,)
이장훈님의 스팀밀크가 완성되었습니다. (첨가물 : 우유, 시럽,)
이여진님의 바닐라라떼가 완성되었습니다. (첨가물 : 에스프레소, 우유, 바닐라시럽,)

singleton 패턴

- singleton 패턴을 사용한 프로그램 - Thread 사용, 동기화 사용
 - 결과가 섞여서 출력되는 문제를 해결하기 위해 getInstance() 메서드를 동기화

코드 7-68

EspressoMachine 클래스

```
public class EspressoMachine {
    public void Brew(String Name, String Beverage) {
        try {
            System.out.println(Name + "님의 "+Beverage+"를 내립니다.");
            Thread.sleep(100);
            System.out.println(Name+ "님의"+Beverage+"가 완성되었습니다.");
        }
        catch(Exception e) {
            e.printStackTrace();
        }
    }
}
```

코드 7-69

User 클래스

```
public class User extends Thread {
    public String Name;
    public String Beverage;
    public User(String Name, String Beverage) {
        this.Name = Name;
        this.Beverage = Beverage;
    }
    public void run() {
        EspressoMachine EM = EspressoMachine.getInstance();
        EM.Brew(Name, Beverage);
    }
}
```

코드 7-70

Main 클래스

[코드 7-67]의 Main 클래스와 동일

실행 결과

에스프레소 머신 생성
이장훈님의 스팀밀크를 내림.
우유를 추가.
시럽을 추가.
이장훈님의 스팀밀크가 완성됨.(첨가물: 우유,시럽,)

이여진님의 바닐라라떼를 내림.
에스프레소를 추가.
우유를 추가.
바닐라시럽을 추가.
이여진님의 바닐라라떼가 완성됨.(첨가물: 에스프레소,우유,바닐라시럽,)
김치수님의 아메리카노를 내림.
에스프레소를 추가.
물을 추가.
김치수님의 아메리카노가 완성됨.(첨가물: 에스프레소,물,)

singleton 패턴

- singleton 패턴을 사용한 프로그램 - Thread 제거, 동기화 사용
 - 결과가 정상으로 나오는 것을 확인했기 때문에 스레드를 제거

코드 7-71

EspressoMachine 클래스 - singleton 패턴을 사용(Thread 제거, synchronized 사용)

```
public class EspressoMachine {
    public static EspressoMachine EM;
    private EspressoMachine() {System.out.println("에스프레소 머신 생성");}
    private String[] vanillaLatte = {"에스프레소", "우유", "바닐라시럽"};
    private String[] americano = {"에스프레소", "물"};
    private String[] stimilk = {"우유", "시럽"};
    private String condiments = "";
    public synchronized static EspressoMachine getInstance() {
        if(EM == null) {
            EM = new EspressoMachine();
        }
        return EM;
    }
    public synchronized void Brew(String Name, String Beverage) {
        System.out.println(Name + "님이 " + Beverage + "를 내립니다.");
        Blend(Beverage);
        System.out.println(Name+"님의 "+Beverage+"가 완성. (첨가물: "+condiments+"");
        condiments = "";
    }
    public void Blend(String Beverage) {
        if (Beverage.equals("바닐라라떼")) {
            for (int i = 0; i < 3; i++) {
                System.out.println(vanillaLatte[i] + "를 추가.");
                condiments = new StringBuffer(condiments).append(vanillaLatte[i]).append(", ").toString();
            }
        }
    }
}
```

=====

singleton 패턴

- singleton 패턴을 사용한 프로그램 - Thread 제거, 동기화 사용
 - 결과가 정상으로 나오는 것을 확인했기 때문에 스레드를 제거

코드 7-72

User 클래스

```
public class User {  
    public String Name;  
    public String Beverage;  
    public User(String Name, String Beverage) {  
        this.Name = Name;  
        this.Beverage = Beverage;  
    }  
    public void make() {  
        EspressoMachine EM = EspressoMachine.getInstance();  
        EM.Brew(Name, Beverage);  
    }  
}
```

코드 7-73

Main 클래스

```
public class Main {  
    public static void main(String args[]) {  
        User user1 = new User("이장훈", "스팀밀크");  
        user1.make();  
        User user2 = new User("이여진", "바닐라라떼");  
        user2.make();  
        User user3 = new User("김치수", "아메리카노");  
        user3.make();  
    }  
}
```

실행 결과

에스프레소 머신 생성
이장훈님이 스팀밀크를 내립니다.
우유를 추가.
시럽을 추가.
이장훈님의 스팀밀크가 완성. (첨가물: 우유,시럽,)
이여진님이 바닐라라떼를 내립니다.
에스프레소를 추가.
우유를 추가.
바닐라시럽을 추가.
이여진님의 바닐라라떼가 완성. (첨가물: 에스프레소,우유,바닐라시럽,)
김치수님이 아메리카노를 내립니다.
에스프레소를 추가.
물을 추가.
김치수님의 아메리카노가 완성. (첨가물: 에스프레소,물,)

singleton 패턴

- singleton 패턴을 사용한 프로그램 - Thread 제거, 동기화 사용
 - ① 생성자를 private으로 선언
 - ② 객체 생성을 static으로 선언
 - ③ getInstance()를 만들어 외부에서 사용하도록 함
 - ④ getInstance() 메서드를 동기화

singleton 패턴

- singleton 패턴을 사용한 프로그램 (객체 생성을 처음부터)
 - synchronized 사용은 두 thread가 getInstance() 메서드를 동시에 실행시키는 것을 방지
 - 동기화를 하면 속도가 떨어지는 문제가 발생하므로 public static EspressoMachine getInstance()처럼 처음부터 생성하여 해결
 - 속도가 중요하지 않다면 public synchronized static EspressoMachine getInstance()처럼 getInstance() 메서드를 동기화해
 - 메서드의 동시 실행을 막음

코드 7-74

EspressoMachine 클래스 - singleton 패턴을 사용(처음부터 객체 생성)

```
public class EspressoMachine {  
    public static EspressoMachine EM = new EspressoMachine();  
    // 동일  
    public static EspressoMachine getInstance() {  
        return EM;  
    }  
    public synchronized void Brew(String Name, String Beverage) {  
        //동일  
    }  
    public void Blend(String Beverage) {  
        // 동일  
    }  
}
```

singleton 패턴

- singleton 패턴을 사용한 프로그램 - DCL 사용
 - DCL을 구현하는 방법은 volatile을 사용

코드 7-75

singleton 패턴을 사용한 프로그램(DCL 사용)

```
public class EspressoMachine {
    public volatile static EspressoMachine EM;
    private EspressoMachine() {System.out.println("에스프레소 머신 생성");}
    private String[] vanillaLatte = {"에스프레소", "우유", "바닐라시럽"};
    private String[] americano = {"에스프레소", "물"};
    private String[] stimilk = {"우유", "바닐라시럽"};
    private String condiments = "";
    public static EspressoMachine getInstance() {
        if(EM == null) {
            synchronized(EspressoMachine.class) {
                if(EM == null) {
                    EM = new EspressoMachine();
                }
            }
        }
        return EM;
    }
    public synchronized void Brew(String Name, String Beverage) {
        System.out.println(Name + "님이 " + Beverage + "를 내립니다.");
        Blend(Beverage);
        System.out.println(Name+"님의 "+Beverage+"가 완성. (첨가물: "+condiments+"");
        condiments = "";
    }
}
```

=====

singleton 패턴

- singleton 패턴을 사용한 프로그램 - DCL 사용

실행 결과

에스프레소 머신 생성
이장훈님이 스팀밀크를 내립니다.
우유를 추가.
시럽을 추가.
이장훈님의 스팀밀크가 완성. (첨가물: 우유,시럽,)
이여진님이 바닐라라떼를 내립니다.
에스프레소를 추가.
우유를 추가.
바닐라시럽을 추가.
이여진님의 바닐라라떼가 완성. (첨가물: 에스프레소,우유,바닐라시럽,)
김치수님이 아메리카노를 내립니다.
에스프레소를 추가.
물을 추가.
김치수님의 아메리카노가 완성. (첨가물: 에스프레소,물,)

END

